

LED Lab 5

Welcome to the fifth lab in our series on Light-Emitting Diodes (LEDs)!

Up until now, our Arduino projects have been "output-only." The microcontroller followed a set of instructions blindly, blinking lights regardless of what was happening in the outside world. In this lab, we bridge the gap between the digital world and the physical world by adding an Input.

The primary objective of this lab is for students to:

- **Understand the difference between Output (sending voltage) and Input (reading voltage)**
- **Use the `digitalRead()` function to detect physical button presses.**
- **Implement "State Change Detection" to ensure one button press equals one action.**
- **Use a variable to track the current "Mode" or state of the system.**

Lab 5 – Mode Selector Switch Lab

In previous labs, if we wanted to change the LED pattern from "S.O.S" to "Alternating," we had to rewrite the code and re-upload it. In this lab, we write one program that contains multiple patterns. We use a variable called mode to remember which pattern is currently active. When the button is pressed, the program adds 1 to the mode variable, causing the Arduino to switch to the next pattern instantly

Enter the Code

Open your Arduino IDE and type in the code below.

```
// --- Hardware Definitions ---  
  
int LED_1 = 2;  
  
int LED_2 = 4;  
  
int LED_3 = 6;  
  
int LED_4 = 8;  
  
const int buttonPin = 10; // The pin that the button is attached to
```

```

// --- State Variables ---

int mode = 0;      // 0=OFF, 1=ALL ON, 2=ALTERNATING

int buttonState = 0;  // Current reading from the input pin

int lastButtonState = 0; // Previous reading (for edge detection)


void setup() {

    // Set LED pins as outputs

    pinMode(LED_1, OUTPUT);

    pinMode(LED_2, OUTPUT);

    pinMode(LED_3, OUTPUT);

    pinMode(LED_4, OUTPUT);


    // Set the button pin as an input

    pinMode(buttonPin, INPUT);
}


void loop() {

    // 1. READ INPUT: Check if button is pressed

    buttonState = digitalRead(buttonPin);


    // 2. CHECK FOR CLICK: Only act when button changes from LOW to HIGH

    if (buttonState == HIGH && lastButtonState == LOW) {

        mode++; // Go to next mode


        // Reset to mode 0 if we go past the last mode

        if (mode > 2) {

            mode = 0;

        }

    }

}

```

```
    delay(100); // Small delay to prevent "bouncing" (reading one press as two)
}

// Save the current state as the last state for the next loop
lastButtonState = buttonState;

// 3. EXECUTE MODE
if (mode == 0) {
    // Mode 0: All OFF
    digitalWrite(LED_1, LOW); digitalWrite(LED_2, LOW);
    digitalWrite(LED_3, LOW); digitalWrite(LED_4, LOW);
}
else if (mode == 1) {
    // Mode 1: All ON
    digitalWrite(LED_1, HIGH); digitalWrite(LED_2, HIGH);
    digitalWrite(LED_3, HIGH); digitalWrite(LED_4, HIGH);
}
else if (mode == 2) {
    // Mode 2: Alternating
    digitalWrite(LED_1, HIGH); digitalWrite(LED_2, HIGH);
    digitalWrite(LED_3, LOW); digitalWrite(LED_4, LOW);
    delay(200);
    digitalWrite(LED_1, LOW); digitalWrite(LED_2, LOW);
    digitalWrite(LED_3, HIGH); digitalWrite(LED_4, HIGH);
    delay(200);
}
}
```

Mini Lab: Mode Selector Switch Lab (Code Overview)

Hardware Definitions

const int buttonPin = 10;

This line tells the Arduino that our pushbutton is connected to Digital Pin 12. We use `const` because this pin number will never change while the program is running.

Setup()

pinMode(buttonPin, INPUT);

Just like we set LEDs to OUTPUT (to send power out), we must set the button pin to INPUT. This configures the pin to "listen" for voltage coming in.

Void loop() – Reading the Button

buttonState = digitalRead(buttonPin);

The `digitalRead()` function checks the voltage on Pin 12.

- If the button is pressed (connected to 5V), it returns **HIGH**.
- If the button is not pressed (pulled to GND), it returns **LOW**.

State Change Detection (The Logic)

if (buttonState == HIGH && lastButtonState == LOW)

This is the most critical logic in the lab. The Arduino loop runs thousands of times per second. If we just said "If button is HIGH, change mode," the mode would cycle hundreds of times in the split second your finger was on the button.

We check for the exact moment the button **transitions** from NOT PRESSED (LOW) to PRESSED (HIGH). This ensures the mode changes only once per click.

The Mode Variable

mode++;

if (mode > 2) { mode = 0; }

This logic creates a cycle.

- Start at 0.
- Press button -> mode becomes 1.
- Press button -> mode becomes 2.
- Press button -> mode becomes 3, but the if statement immediately resets it to 0.

Mini Lab: Mini Lab: Mode Selector Switch Lab (Hardware Overview)

Since this project uses the same four LEDs as the previous lab, the hardware setup remains identical. We still require four separate current-limiting resistors.

Component	Quantity	Key Function
Arduino UNO R3	1	The main programmable controller.
Breadboard	1	Used to connect all components without soldering.
5mm LED	4	Converts electrical energy into light.
100-ohm Resistor	4	Limits the current to protect each LED from burning out.
Jumper Wires	13	Used to connect the four LEDs and resistors to the Arduino pins.
10k-ohm Resistor	1	Pull-Down Resistor keeps the input pin at 0V (LOW) when the button is not pressed.
Pushbutton	1	Connects 5V to the input Pin when pressed.

Connecting Everything (Step by Step Instructions)

Place the LEDs: Insert the four 5mm LEDs into the breadboard. Ensure the longer leg (Anode) is oriented correctly.

Attach the Resistors: Connect one lead of a 100 Ω resistor to the shorter leg (Cathode) of each LED.

Connect to Ground: Connect the free ends of all four resistors to the **GND** rail on the breadboard, and connect that rail to the GND pin on the Arduino.

Connect to Digital Pins: Use jumper wires to connect the longer leg (Anode) of each LED to the Arduino as follows:

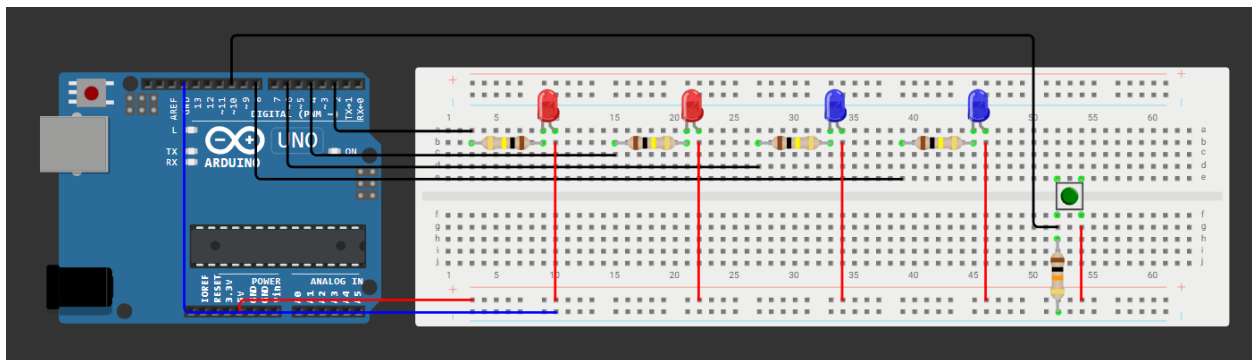
- **LED 1:** Digital Pin 2
- **LED 2:** Digital Pin 4
- **LED 3:** Digital Pin 6
- **LED 4:** Digital Pin 8

Place the Button: Insert the pushbutton into the breadboard so it straddles the center gap (the "ravine").

Power Side: Connect one leg of the button directly to **5V**.

Signal Side (The Junction): Locate the leg diagonally opposite to the one you just connected to 5V. On this specific breadboard row, you need to make **two** connections:

- **Connection A:** Insert a wire connecting this row to **Pin 12** on the Arduino.
- **Connection B:** Insert one leg of the **10k-ohm resistor** into this same row. Connect the other leg of the resistor to **GND**.



Upload and Observe:

- Upload the code. The LEDs should start OFF (Mode 0).
- Press the button once: All LEDs should turn ON (Mode 1).
- Press the button again: The LEDs should start ALTERNATING (Mode 2).
- Press the button a third time: The LEDs turn OFF, restarting the cycle.